

Reflection and Attributes



SoftUni Team
Technical Trainers



**SoftUni
Foundation**



Software University

<http://softuni.bg>

Table of Contents

1. What? Why? Where? When?
2. Reflection API
 - Type Class
 - Reflecting Fields
 - Reflecting Constructors
 - Reflecting Methods
3. Attributes
 - Applying Attributes to Code Elements
 - Built-in Attributes
 - Defining Attributes



sli.do

#fund-csharp



Reflection

What? Why? Where? When?

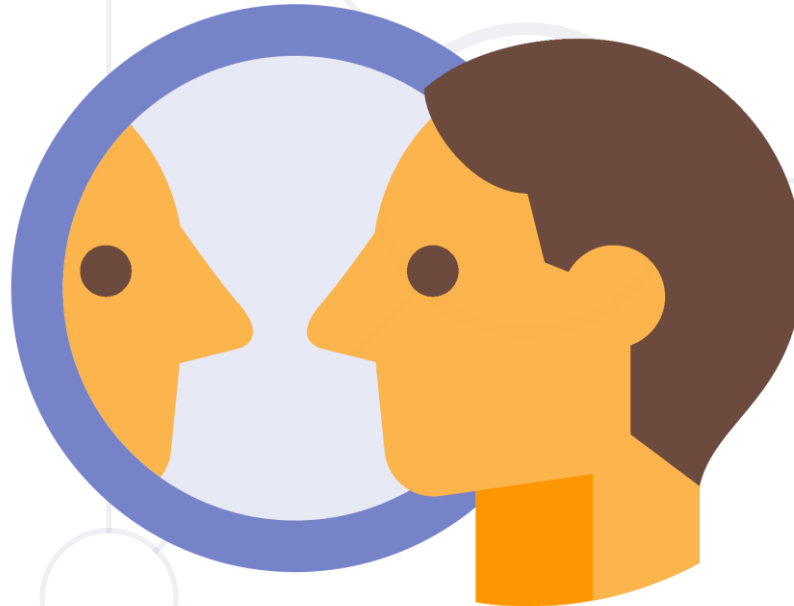
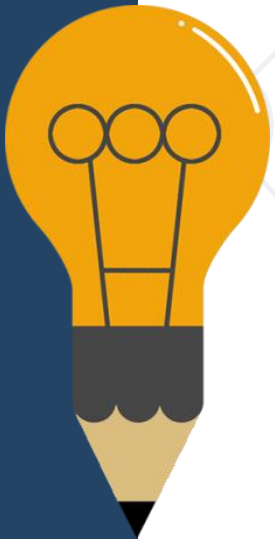
What is Metaprogramming?

- **Programming technique**, in which computer programs have the ability to treat other **programs as their data**
- Programs can be designed to:
 - **Read**
 - **Generate**
 - **Analyze**
 - **Transform**
 - **Modify itself** while **running**



What is Reflection?

- The ability of a programming language to be its **own metalanguage**
- Programs can examine information about **themselves**



When to Use Reflection?

- Whenever we want:
 - Code to become more **extendible**
 - To **reduce code** length significantly
 - Easier **maintenance**
 - Easier **testing**



When not to Use Reflection?

- If it is **possible** to **perform** an operation **without** using **reflection**, then it is preferable to **avoid using it**
- Cons from using Reflection
 - **Performance** overhead
 - **Security** restrictions
 - Exposure of **internals**





Reflection API

Reflecting Class and Members

- Primary way to access **metadata**
- Obtained at **compile time**, if you know its **name**:

```
Type myType = typeof(ClassName);
```

- Can be obtained at **runtime**, if the name is **unknown**:

```
Type myType = Type.GetType("Namespace.ClassName");
```

You need fully qualified class name as string

- Obtain Class name

- Fully qualified class name - Type.FullName

```
string fullName = typeof(SomeClass).FullName;
```

- Class name without the namespace - Type.Name

```
string simpleName = typeof(SomeClass).Name;
```

- Obtain **base type**

```
Type baseType = testClass.BaseType;
```

- Obtain **interfaces**

```
Type[] interfaces = testClass.GetInterfaces();
```

- All the **interfaces that the class implements** are returned
 - Even interfaces from **base classes**

- **Activator.CreateInstance**
 - Creates an instance of a type by invoking the constructor that **matches** the specified **arguments**

```
var sbType = Type.GetType("System.Text.StringBuilder");  
StringBuilder sbInstance =  
    (StringBuilder) Activator.CreateInstance(sbType);  
StringBuilder sbInstCapacity = (StringBuilder)Activator  
    .CreateInstance(sbType, new object[] { 10 });
```

- Obtain public fields

```
FieldInfo field = type.GetField("name");  
FieldInfo[] publicFields = type.GetFields();
```

- Obtain all fields

```
FieldInfo[] allFields = type.GetFields(  
    BindingFlags.Static |  
    BindingFlags.Instance |  
    BindingFlags.Public |  
    BindingFlags.NonPublic);
```

- Each modifier is **a flag bit** that is either set or cleared
- Check **access modifier** of a **member** of the class

<code>field.IsPrivate</code>	<i>//private</i>
<code>field.IsPublic</code>	<i>//public</i>
<code>field.IsNonPublic</code>	<i>//everything but public</i>
<code>field.IsFamily</code>	<i>//protected</i>
<code>field.IsAssembly</code>	<i>//internal</i>

- The **BindingFlags** enum specifies what kinds of types we are looking up

```
FieldInfo[] allFields =  
    type.GetFields(BindingFlags.NonPublic);
```

- Can be combined with bitwise OR (| operator):

```
FieldInfo[] allFields = type.GetFields(  
    BindingFlags.Public |  
    BindingFlags.NonPublic);
```

Returns both public
and nonpublic fields

- Get **public** field **name and type**

```
FieldInfo field = type.GetField("fieldName");  
string fieldName = field.Name;  
Type fieldType = field.FieldType;
```

- Use **BindingFlags** to specify access modifiers, if the field is not public, otherwise **GetField** returns **null**

Changing a Field's State

```
Type testType = typeof(Test);  
Test testInstance =  
    (Test) Activator.CreateInstance(testType);  
FieldInfo field = testType.GetField("testInt");  
  
field.SetValue(testInstance, 5);  
int fieldValue =  
    (int)field.GetValue(testInstance);
```

Changes the
object's state

- Obtain **constructors**

```
ConstructorInfo[] publicCtors =  
    type.GetConstructors();
```

- Obtain **all non static constructors**

```
ConstructorInfo[] allNonStaticCtors =  
    type.GetConstructors(  
        BindingFlags.Instance |  
        BindingFlags.Public |  
        BindingFlags.NonPublic);
```

- Obtain a certain constructor

```
ConstructorInfo constructor =  
    type.GetConstructor(new Type[] { typeof(...) } );
```

- Get constructor parameters

```
ParameterInfo[] parameterTypes =  
    constructor.GetParameters();
```

- Instantiating objects using a specific constructor

```
SomeClass instanceClass =  
    (SomeClass)constructor.Invoke(  
        new object[] { "gosho", 5 } );
```

Supply positional parameters
in an object array

- Obtain all **public** methods

```
MethodInfo[] publicMethods = type.GetMethods();
```

- Obtain a **certain** method

```
MethodInfo specialMethod =  
    type.GetMethod("MethodName", new  
Type[0]);  
MethodInfo overloadMethod = type.GetMethod(  
    "MethodName", new [] { typeof(string)  
});
```

- Obtain method **parameters** and **return type**

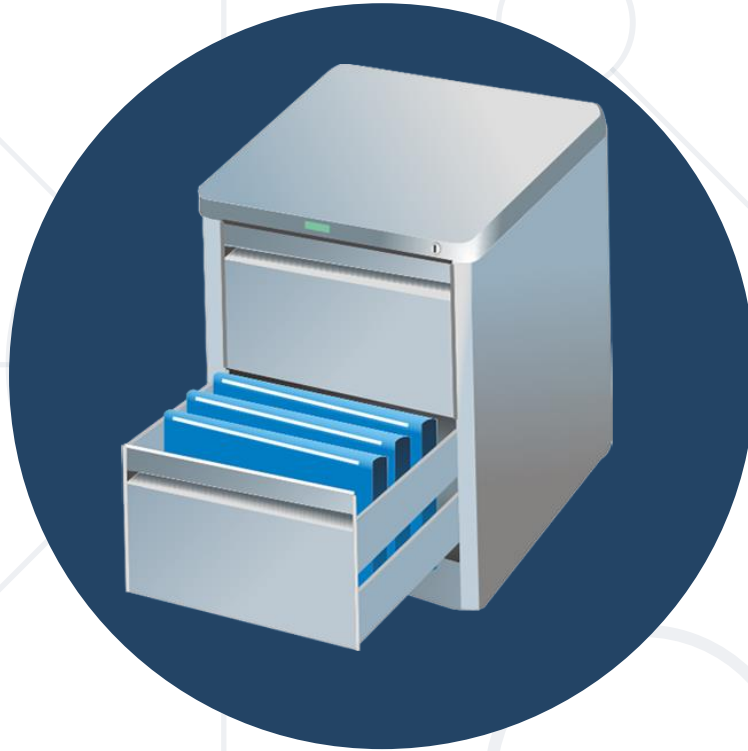
```
ParameterInfo[] methodParameters =  
    overloadMethod.GetParameters();  
Type returnType = overloadMethod.ReturnType;
```

- Invoke** methods

```
overloadMethod.Invoke(instanceClass, new object[] {  
    "hi!" });
```

Target object
instance

Parameters for
the method



Attributes

Data About Data

Attributes

- **Data holding** class
- **Describes** parts of your code
- Applied to:
 - **Classes, Fields, Methods**, etc.



```
[Obsolete]
public void DeprecatedMethod()
{
    Console.WriteLine("Deprecated!");
}
```


- Generate **compiler messages** or **errors**

[Obsolete]

```
public enum Coin // Enum 'Coin' is obsolete
```

- Tools, which rely on attributes:
 - **Code generation** tools
 - **Documentation generation** tools
 - **Testing** Frameworks
- Runtime – **ORM, Serialization** etc.

- Attribute's name is surrounded by **square brackets: []**
 - Placed before their target declaration

```
[Flags] // System.FlagsAttribute
public enum FileAccess
{
    Read = 1,
    Write = 2,
    ReadWrite = Read | Write
}
```

- **[Flags]** attribute indicates that the enum type can be treated like a set of bit flags, stored as a single integer

Attributes with Parameters (2)

- Attributes can accept **parameters** for their constructors and public properties

```
[DllImport("user32.dll", EntryPoint="MessageBox")]  
public static extern int ShowMessageBox(int hWnd,  
    string text, string caption, int type);  
...  
ShowMessageBox(0, "Some text", "Some caption", 0);
```

- The [DllImport] attribute refers to:
 - System.Runtime.InteropServices.DllImportAttribute**
 - "**user32.dll**" is passed to the constructor
 - "**MessageBox**" value is assigned to **EntryPoint**

Set a Target to an Attribute

- Attributes can specify their **target** declaration:

```
// target "assembly"
[assembly: AssemblyTitle("Attributes Demo")]
[assembly: AssemblyCompany("DemoSoft")]
[assembly: AssemblyProduct("Enterprise Demo Suite")]
[assembly: AssemblyVersion("2.0.1.37")]

[Serializable] // [type: Serializable]
class TestClass
{
    [NonSerialized] // [field: NonSerialized]
    private int status;
}
```

Custom Attributes Requirements

- Must **inherit** the **System.Attribute** class
- Their **names** must end with "**Attribute**"
- Possible **targets** must be defined via **[AttributeUsage]**
- Can define **constructors** with parameters
- Can define public **fields** and **properties**

Problem: Create Attribute

- Create an attribute **Author** with a **string** element called **name** that:
 - Can be used over **classes and methods**
 - Allow multiple attributes of same type

```
[Author("Ventsi")]
public class StartUp
{
    [Author("Gosho")]
    static void Main(string[] args)
    { ... }
}
```

Check your solution here: <https://judge.softuni.bg/Contests/710/Reflection-and-Attributes-Lab>

Solution: Create Attribute

```
[AttributeUsage(AttributeTargets.Class |
                AttributeTargets.Method,
                AllowMultiple = true)]
public class SoftUniAttribute : Attribute
{
    public SoftUniAttribute(string name)
    {
        this.Name = name;
    }
    public string Name { get; set; }
}
```

Check your solution here: <https://judge.softuni.bg/Contests/710/Reflection-and-Attributes-Lab>

Problem: Coding Tracker

- Create a class **Tracker** with a method:
 - **void PrintMethodsByAuthor()**
- Print to the console authors for all methods
 - Use **SoftUni attribute** and **reflection**



Check your solution here: <https://judge.softuni.bg/Contests/710/Reflection-and-Attributes-Lab>

Solution: Coding Tracker

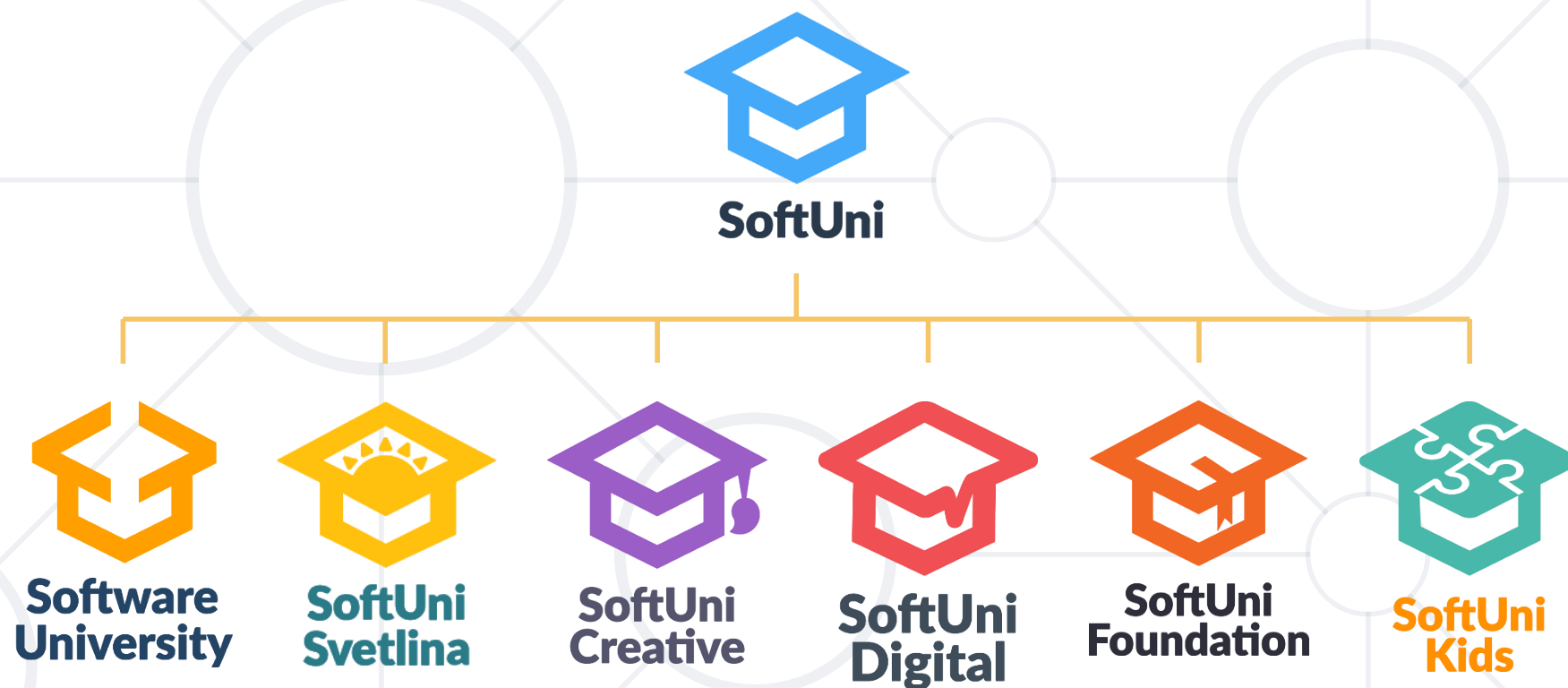
```
var type = typeof(Startup);
var methods =
    type.GetMethods(
        BindingFlags.Instance | BindingFlags.Public |
        BindingFlags.Static);
foreach (var method in methods) {
    var attributes = method.CustomAttributes
        .Where(n => n.AttributeType == typeof(SoftUniAttribute))
    foreach (SoftUniAttribute attr in attributes) {
        Console.WriteLine("{0} is written by {1}",
            method.Name, attr.Name);
    }
}
```

Check your solution here: <https://judge.softuni.bg/Contests/710/Reflection-and-Attributes-Lab>

- **Reflection:**
 - Allows us to get **information about types**
 - Allows us to dynamically **call methods**, **get/set** values, etc.
- **Attributes** allow adding metadata in classes / types / etc.
 - Built-in attributes
 - Custom attributes
 - Can be accessed at runtime



Questions?



SoftUni Diamond Partners



XSsoftware



SBTech
we know sports



telenor



SoftwareGroup
doing it right

NETPEAK



SmartIT



Postbank

Решения за твоето утре

**SUPER
HOSTING**
.BG

INDEAVR

Serving the high achievers



INFRAGISTICS®

LIEBHERR



æternity



codexio

SoftUni Organizational Partners



OneBit
SOFTWARE



 **codexio**

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education and Employment Opportunities
 - softuni.bg
- Software University Foundation
 - <http://softuni.foundation/>
- Software University @ Facebook
 - facebook.com/SoftwareUniversity
- Software University Forums
 - forum.softuni.bg



- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license

